NATIONAL ECONOMICS UNIVERSITY
SCHOOL OF TECHNOLOGY



REPORT

TIME SERIES ANALYSIS AND FORECASTING

Topic:

ST-MEM-BASED ECG CLASSIFICATION FOR ABNORMAL RHYTHM DETECTION USING MIT-BIH ECG RECORDINGS

Group 4 members: Vu Quoc Huy – 11247301
Tran Tuan Anh – 11247260
Nguyen Tuan Anh – 11247264
Tran Minh Hoang – 11247292
Class: AI66A
Supervisor: Dr. Trong-Nghia Nguyen

Hanoi, May 2026

TABLE OF CONTENTS

LIST OF FIGURES iii

LIST OF TABLES iv

1 Introduction 1

2 Related Work 3

3 Dataset and Preprocessing 5

3.1 Dataset 5

3.2 Preprocessing 5

3.2.1 Objective of Preprocessing 5

3.2.2 Input and Output of the Pipeline 6

3.2.3 Baseline Wander Removal 6

3.2.4 Power-Line Noise Removal using Notch Filter 7

3.2.5 Band-pass Filtering 0.5–40 Hz 8

3.3 Label and Dataset Split 9

4 Method 11

4.1 Sliding Window 11

4.1.1 Feature Extraction 11

4.1.2 Sample Structure 12

4.1.3 Feature Counts, Distributions and Correlation 12

4.1.4 Models Used 13

4.2 CNN 13

4.2.1 Overview of 1D CNN Model 13

4.2.2 Convolutional Feature Extraction 14

4.2.3 Global Average Pooling 1D 15

4.2.4 Classification Head 16

4.2.5 Training Strategy 16
4.3 ST-MEM 16
4.3.1 Motivation 16
4.3.2 Pretrain ST-MEM 17
4.3.3 Downstream Task on the MIT-BIH Dataset 20

5 Experiments and Results **24**
5.1 Sliding Window 24
5.2 CNN 25
5.3 ST-MEM 26
5.4 Comparison Design 27

6 Discussion, Limitations, and Conclusion **29**
6.1 Discussion 29
6.2 Limitations 29
6.3 Conclusion 30

REFERENCES **31**

LIST OF FIGURES

3.1 Comparison between raw ECG and preprocessed ECG signal 9

4.1 Overall architecture of the proposed 1D CNN model 14

4.2 ST-MEM architecture 16

5.1 Confusion matrix comparison of the best 4.5s and 9s sliding-window models 25

LIST OF TABLES

3.1 Dataset size after sliding-window segmentation 10

4.1 Example of window-level features 12

4.2 Example of frame-level features 12

4.3 Feature count in the sliding-window feature set 13

5.1 Sliding-window classification results 24

5.2 CNN classification results 25

5.3 ST-MEM classification results 26

5.4 Comparison between supervised training and ST-MEM on PTB-XL . . 27

5.5 Best model comparison among Sliding Window, CNN, and ST-MEM on the test set 27

1. Introduction

Electrocardiogram (ECG) signals are one of the most important sources of data in cardiovascular medicine. ECG is used to monitor the electrical activity of the heart and support the detection of abnormal cardiac rhythms [1]. Since ECG records changes in heart electrical potential over time, the signal contains valuable information about waveform morphology, cardiac cycles, and abnormal patterns that may be related to arrhythmia.

In biomedical signal processing research, the MIT-BIH Arrhythmia Database is one of the most widely used benchmark datasets. It contains ECG recordings with expert annotations, making it suitable for developing and evaluating heartbeat and arrhythmia classification models [2–4].

However, ECG classification is not a simple task. Raw ECG signals often contain several types of noise, such as baseline wander, power-line interference, and high-frequency noise. These noise components may distort the signal baseline or hide important features such as the QRS complex and R-peaks. In addition, ECG is a one-dimensional time-series signal. Important information is not only located at individual signal points but also in waveform shapes and temporal variations across the signal segment. Therefore, before applying a classification model, a suitable preprocessing pipeline is required to improve signal quality and preserve meaningful features for classification.

Previous studies on ECG classification have used handcrafted feature-based methods and deep learning models to detect abnormal cardiac patterns. However, each approach has certain limitations. Handcrafted feature-based methods depend heavily on the feature design process, require domain knowledge, and may miss complex abnormal patterns in ECG signals. Deep learning models can learn features automatically from data, but their effectiveness still depends on input signal preparation, segmentation strategy, and the ability to handle class imbalance between Normal and Abnormal samples.

In this project, an important challenge is how to divide the ECG signal into suitable input windows for classification. If the window is too short, the model may not receive enough temporal context to distinguish between normal and abnormal rhythms. If the window is too long, the number of training samples decreases and computational cost may increase. Moreover, ECG datasets often contain a large imbalance between normal and abnormal samples, which may cause the model to be biased toward the majority class if no appropriate training strategy is applied.

Based on these challenges, this report aims to build a clear and explainable ECG classification pipeline on the MIT-BIH Arrhythmia Database. Instead of focusing only on final prediction results, the study considers the entire workflow, including raw ECG prepro-

cessing, sliding-window segmentation, feature-based modelling, 1D CNN classification, and ST-MEM-based representation learning. This workflow is designed to preserve important ECG characteristics after preprocessing and provide sufficient temporal context for the model to learn meaningful abnormal patterns.

In this study, a 1D Convolutional Neural Network is used because it is suitable for one-dimensional time-series data such as ECG. Convolutional kernels can slide along the temporal axis and learn features such as QRS morphology, slopes, amplitudes, peak locations, and local waveform distortions without requiring handcrafted feature design [5–7]. In addition, this report also considers ST-MEM as a comparison method to evaluate whether a representation learning approach can exploit ECG signal representations more effectively for classification [8, 9].

The report focuses on the following research questions:

1. Does the preprocessing pipeline, including baseline wander removal, power-line noise removal, and band-pass filtering, improve ECG signal quality while preserving important features for classification?
2. How does sliding-window segmentation with different window lengths affect classification performance between Normal and Abnormal ECG windows?
3. Can the 1D CNN model learn meaningful features from preprocessed ECG signals and achieve effective binary classification performance?
4. When evaluated under the same data setup, how do 1D CNN and ST-MEM differ in performance, and which method is more suitable for ECG classification in this study?
5. Which factors in the data processing pipeline and model design have the strongest influence on ECG classification quality?

2. Related Work

ECG classification studies can generally be divided into three main directions: handcrafted feature-based methods, supervised deep learning methods, and more recent self-supervised learning methods for ECG representation learning.

In traditional studies, ECG signals are usually preprocessed first. Then, handcrafted features such as waveform morphology, RR intervals, or wavelet-based coefficients are extracted and passed into machine learning classifiers such as Support Vector Machine, Random Forest, or other models. For example, Emanet used Discrete Wavelet Transform to extract ECG features and combined them with Random Forest for heartbeat classification [10]. Other studies also used wavelet features with Support Vector Machine to identify abnormal heartbeat patterns [11].

The main advantage of handcrafted feature-based methods is that they are relatively interpretable. Each feature usually has a clear meaning, such as rhythm variation, amplitude change, or waveform shape. These methods also do not require very large datasets. However, their performance depends strongly on the quality of feature engineering. If the selected features do not fully capture ECG morphology or temporal variations, the classifier may fail to detect complex abnormal patterns.

To reduce the dependence on handcrafted features, recent studies have increasingly used deep learning, especially Convolutional Neural Networks. CNN-based models can automatically learn important features directly from ECG signals or from ECG representations. Jun et al. converted ECG beats into two-dimensional images and used a 2D Convolutional Neural Network for arrhythmia classification on the MIT-BIH dataset [12]. This approach showed that CNN models can achieve strong classification performance without relying completely on manually designed features.

For ECG data, 1D CNN is particularly suitable because it preserves the original one-dimensional temporal structure of the signal. Kiranyaz et al. proposed a real-time patient-specific ECG classification method using 1D Convolutional Neural Networks [5]. Acharya et al. developed a deep CNN model to classify heartbeats and showed that stacked convolutional layers can learn hierarchical ECG representations [6]. Yildirim et al. used a deep convolutional neural network with long-duration ECG signals for arrhythmia detection, showing the importance of temporal context in ECG classification [7]. These studies indicate that CNN-based models can learn ECG morphology, slope, amplitude, and local deformation patterns directly from signal windows.

Although CNN-based models can learn features automatically, their performance still depends on preprocessing quality, segmentation strategy, class balance, and architecture

design. Therefore, it is important to evaluate not only the model itself but also the complete data pipeline used to prepare input samples.

Besides supervised deep learning, self-supervised learning has recently become an important direction for ECG representation learning. In self-supervised learning, a model is first trained to learn general representations from unlabeled or weakly labelled signals and is then adapted to a downstream classification task. Na et al. proposed ST-MEM, or Spatio-Temporal Masked Electrocardiogram Modeling, which learns ECG representations by masking parts of ECG signals and reconstructing the missing information [8]. The official implementation of ST-MEM is also provided by VUNO, which supports reproducibility and reference for model structure and training settings [9].

ST-MEM has the advantage of using unlabeled data and learning richer ECG representations than training from scratch. However, ST-MEM was originally designed mainly for 12-lead ECG signals, while the MIT-BIH Arrhythmia Database used in this report contains two-channel ECG recordings [2,3]. Therefore, when applied in this project, ST-MEM needs to be considered carefully because the downstream input structure differs from the original pretraining setting.

Based on the reviewed studies, this report selects 1D CNN as the main deep learning model because it can learn directly from one-dimensional ECG windows after preprocessing. The report also includes a feature-based sliding-window approach and uses ST-MEM as a comparison method. This allows the study to compare handcrafted feature modeling, supervised deep learning, and pre-trained ECG representation learning in the context of Normal and Abnormal ECG classification.

3. Dataset and Preprocessing

3.1. Dataset

This project uses the MIT-BIH Arrhythmia Database, accessed through the WFDB library in Python [2–4]. The dataset consists of 48 two-channel ambulatory ECG records. Each record is approximately 30 minutes long and was collected from 47 subjects at Beth Israel Hospital between 1975 and 1979. The sampling frequency is 360 Hz, which provides sufficient temporal resolution for identifying important ECG components such as the QRS complex and R-peaks.

The dataset was selected with a specific sampling strategy. Twenty-three records were selected randomly to represent common ECG distributions, while the remaining twenty-five records were selected to include less frequent but clinically important arrhythmias.

Each MIT-BIH record contains three main file types:

- **Header file (.hea):** contains metadata such as the number of channels, sampling frequency, lead names, and record information.
- **Data file (.dat):** stores the raw ECG signal values in binary format. This file provides the main input signal for machine learning and deep learning algorithms.
- **Annotation file (.atr):** stores expert annotations, including sample positions and heartbeat symbols. These annotations are used as ground truth for training and evaluation.

3.2. Preprocessing

3.2.1. Objective of Preprocessing

Raw ECG signals in the MIT-BIH dataset usually contain several noise components that may affect signal analysis and feature extraction. Common noise types include baseline wander, power-line noise, and high-frequency noise. If they are not processed, these noise components can shift the baseline, create artificial oscillations, or interfere with QRS and R-peak detection [13, 14].

The objective of preprocessing is to clean the ECG signal while preserving important cardiac features, especially the QRS complex and R-peak locations. The preprocessed signal is then used for the next stages, such as sliding-window segmentation, feature extraction, and model classification.

3.2.2. Input and Output of the Pipeline

The input of the preprocessing pipeline is the raw ECG signal read from the MIT-BIH database using WFDB:

$$X_{raw}[n]$$

where:

- $X_{raw}[n]$ is the ECG amplitude at sample index n.
- n is the sample index.
- f_s is the sampling frequency.
- For MIT-BIH, $f_s = 360$ Hz.

The output of the pipeline is the final preprocessed ECG signal:

$$X_{final}[n]$$

The overall preprocessing flow is represented as:

$$X_{raw}[n] \rightarrow X_{detrended}[n] \rightarrow X_{notched}[n] \rightarrow X_{bandpassed}[n] \rightarrow X_{final}[n]$$

3.2.3. Baseline Wander Removal

Baseline wander is a slow drift of the ECG baseline. It is often caused by respiration, body movement, or changes in electrode contact. This component usually appears in the very low-frequency range, often below approximately 0.5 Hz [14]. If it is not removed, the ECG signal may be shifted away from the zero level, which affects amplitude measurement and R-peak detection.

In this project, the baseline is estimated using a two-step median filtering method:

$$b_1[n] = \text{median}_{w_1}(X_{raw}[n])$$

$$b[n] = \text{median}_{w_2}(b_1[n])$$

where $b_1[n]$ is the signal after the first median filter, $b[n]$ is the estimated baseline, w_1 is the first median filter window size, and w_2 is the second median filter window size.

The filter window sizes are selected based on the sampling frequency:

$$w_1 = 0.2 \times f_s$$

$$w_2 = 0.6 \times f_s$$

After estimating the baseline, the detrended ECG signal is calculated as:

$$X_{detrended}[n] = X_{raw}[n] - b[n]$$

After this step, the ECG signal becomes more stable around the zero baseline, while the shape and location of QRS complexes and R-peaks are preserved.

3.2.4. Power-Line Noise Removal using Notch Filter

Power-line noise usually appears at 50 Hz or 60 Hz, depending on the electrical environment and recording device. In ECG preprocessing, notch filtering is commonly used to suppress this narrow-band interference while preserving the main ECG waveform [14].

The frequency to be removed is selected as:

$$f_0 \in \{50, 60\}$$

The notch filter is designed using the selected frequency f_0, the sampling frequency f_s, and the quality factor Q:

$$(b_{notch}, a_{notch}) = \text{iirnotch}(f_0, Q, f_s)$$

where f_0 is the frequency to be removed, Q is the quality factor, f_s is the sampling frequency, and b_{notch}, a_{notch} are the notch filter coefficients.

In this project, the quality factor is:

$$Q = 30$$

The notch-filtered signal is calculated as:

$$X_{notched}[n] = \text{filtfilt}(b_{notch}, a_{notch}, X_{detrended}[n])$$

This step strongly reduces the power-line noise component. The use of zero-phase filtering helps reduce phase delay and avoids shifting R-peak locations.

3.2.5. Band-pass Filtering 0.5–40 Hz

After baseline wander and power-line noise removal, the ECG signal may still contain frequency components outside the useful ECG range. Therefore, a Butterworth band-pass filter is applied to keep the important ECG frequency range and reduce unwanted low-frequency and high-frequency components [13, 14].

$$0.5 \leq f \leq 40 \text{ Hz}$$

The reasons for choosing this range are:

- Components below 0.5 Hz are mainly related to baseline wander.
- Components above 40 Hz are often related to high-frequency noise, EMG artifacts, or rapid motion noise.
- The range 0.5–40 Hz still preserves important ECG waveform components, especially the QRS complex.

Before designing the filter, the Nyquist frequency is calculated as:

$$f_N = \frac{f_s}{2}$$

The cutoff frequencies are normalized as:

$$W_{low} = \frac{0.5}{f_N}$$

$$W_{high} = \frac{40}{f_N}$$

The fourth-order Butterworth band-pass filter is designed as:

$$(b_{bp}, a_{bp}) = \text{butter}(4, [W_{low}, W_{high}], \text{bandpass})$$

where 4 is the filter order, W_{low} is the normalized lower cutoff frequency, W_{high} is the normalized upper cutoff frequency, and b_{bp}, a_{bp} are the band-pass filter coefficients.

The band-pass filtered signal is calculated as:

$$X_{bandpassed}[n] = \text{filtfilt}(b_{bp}, a_{bp}, X_{notched}[n])$$

The final preprocessed ECG signal is:

$$X_{final}[n] = X_{bandpassed}[n]$$

After preprocessing, the ECG signal becomes smoother and more suitable for feature extraction and classification. The QRS complexes and R-peaks remain visible, showing that the preprocessing pipeline improves signal quality without removing important ECG characteristics.

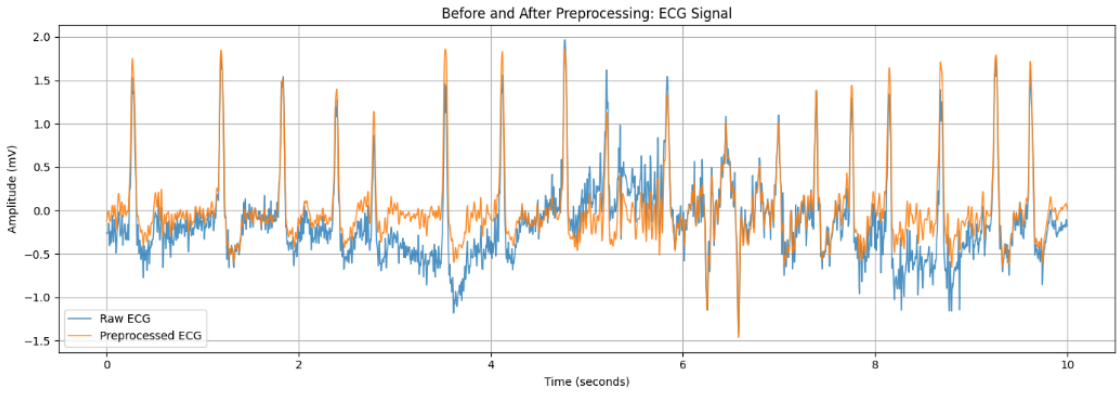


Figure 3.1 Comparison between raw ECG and preprocessed ECG signal

3.3. Label and Dataset Split

After preprocessing the ECG signal, the annotation labels in the MIT-BIH dataset are converted into binary labels for Normal and Abnormal classification.

- **Normal label:** only the annotation symbol `N` is used for the Normal class.
- **Labels not used for classification:** symbols `+`, `x`, `"`, `Q`, and `~` are not treated as direct classification labels. If a window contains only these labels, the window is dropped.
- **Abnormal label:** all remaining valid beat annotations are assigned to the Abnormal class.
- **Dataset split:** records `102`, `103`, `104`, `105`, and `106` are used for testing; records `200`, `208`, `232`, `233`, and `234` are used for validation; all remaining records are used for training.
- **Window versions:** two window lengths are used, 4.5 seconds and 9 seconds. With $f_s = 360$ Hz, the 4.5-second window contains 1620 signal samples, while the 9-second window contains 3240 signal samples.

After applying sliding-window segmentation, the ECG recordings are converted into window-based samples. The dataset size after segmentation is shown in Table 3.1.

Table 3.1 Dataset size after sliding-window segmentation

Window Length	Dataset Split	Number of Windows	Number of Features
4.5s	Train	15,238	86
	Validation	2,004	86
	Test	2,004	86
9s	Train	15,200	165
	Validation	2,000	165
	Test	2,000	165

For the 4.5-second configuration, the training set contains 10,766 Normal windows and 4,472 Abnormal windows. The validation set contains 1,188 Normal windows and 816 Abnormal windows. The test set contains 1,150 Normal windows and 854 Abnormal windows.

4. Method

4.1. Sliding Window

The main purpose of the Sliding Window approach is to extract useful ECG features from each signal window so that traditional machine learning models can use these features to predict whether an ECG window is Normal or Abnormal.

In this approach, each ECG record is divided into fixed-length windows. Two window configurations are used: 4.5 seconds and 9 seconds. A 4.5-second window is divided into six 0.75-second frames, while a 9-second window is divided into twelve 0.75-second frames.

From each window, two groups of features are extracted: frame-level features from each short segment and window-level features from the whole ECG window. After feature extraction, the feature matrix is standardized using StandardScaler, balanced using SMOTE, and reduced using SelectKBest to select the most important features. These preprocessing and feature-selection operations are implemented using common machine learning tools from scikit-learn [15]. SMOTE is used to reduce the effect of class imbalance by generating synthetic minority-class samples [16].

4.1.1. Feature Extraction

Window-level features

Window-level features are extracted from the whole ECG window. These features describe rhythm-related information such as beat count, RR intervals, and heart rate statistics. Heartbeat interval features and ECG morphology features have been widely used in ECG heartbeat classification because they provide useful information about rhythm variation and waveform characteristics [17].

The window-level features include beat count, mean RR, median RR, RR range, normalized RR mean, normalized RR standard deviation, mean heart rate, and heart rate range. For longer windows, RMSSD can also be used to measure short-term variation between consecutive RR intervals.

Frame-level features

Frame-level features are extracted from each 0.75-second frame inside a window. These features describe local signal morphology and short-term waveform changes. The feature groups used in this study are consistent with common time-series feature extraction settings [18].

The frame-level features include amplitude features such as mean, standard deviation,

minimum, maximum, interquartile range, and RMS; local dynamic features such as absolute sum of changes, mean absolute change, line length, and second derivative; distribution shape features such as skewness and kurtosis; and peak-to-peak amplitude.

4.1.2. Sample Structure

Each sample contains two levels of features: window-level features and frame-level features. Window-level features describe the whole ECG window, while frame-level features describe the smaller 0.75-second frames inside the window.

Table 4.1 Example of window-level features

Sample ID	beat_count	mean_RR	median_RR	RR_range	mean_HR	HR_range
Sample_01	11.0	0.82	0.81	0.07	73.68	6.83

Table 4.2 Example of frame-level features

Time Series	feat_mean	feat_std	feat_rms	feat_line_length	feat_mean_abs_change	feat_peak_to_peak_amplitude
TS1	0.6520	0.7337	0.9815	17.4466	0.0649	4.1570
TS2	0.6918	0.7270	1.0036	17.4631	0.0649	3.9469
TS3	0.5963	0.7791	0.9811	17.8045	0.0662	3.7925
TS4	0.6344	0.6391	0.9005	9.1792	0.0341	3.2221
TS5	0.4808	0.7536	0.8939	13.7765	0.0512	3.4903
TS6	0.4624	0.7880	0.9136	15.0051	0.0558	4.1455
TS7	0.6307	0.7588	0.9867	18.5981	0.0691	4.0646
TS8	0.6211	0.7747	0.9929	17.7695	0.0661	3.8682
TS9	0.6505	0.7939	1.0263	17.3652	0.0646	3.8717
TS10	0.5449	0.7684	0.9419	17.6108	0.0655	4.2304
TS11	0.5817	0.7380	0.9396	17.1486	0.0637	3.9879
TS12	0.6263	0.6972	0.9372	16.5183	0.0614	3.7285

4.1.3. Feature Counts, Distributions and Correlation

For the 4.5-second configuration, one window is divided into six frames. Since each frame contains 13 frame-level features and each window contains 8 window-level features, the total number of features is:

$$6 \times 13 + 8 = 86$$

For the 9-second configuration, one window is divided into twelve frames. Since each frame contains 13 frame-level features and each window contains 9 window-level features, including the additional RMSSD feature, the total number of features is:

$$12 \times 13 + 9 = 165$$

Table 4.3 Feature count in the sliding-window feature set

No.	Feature Group	Number of Features	Description
0	Frame-level / frame	13	Features extracted from each 0.75-second frame
1	Window-level	8	RR, HR, and beat statistics
2	Total features for 4.5s	86	Six frames combined with window-level features
3	Total features for 9s	165	Twelve frames combined with window-level features

4.1.4. Models Used

The feature-based models used in this experiment include Random Forest, Extra Trees, and XGBoost [19–21].

- **Random Forest:** an ensemble model that builds multiple decision trees and combines their predictions.
- **Extra Trees:** an ensemble tree-based model that introduces additional randomness when selecting split points.
- **XGBoost:** a gradient-boosting tree model designed for efficient and scalable supervised learning.

4.2. CNN

4.2.1. Overview of 1D CNN Model

The model used in this project is a 1D Convolutional Neural Network. This architecture is suitable for ECG data because ECG is a one-dimensional time-series signal. Important features such as QRS morphology, amplitude, slope, peak location, and local deformation appear along the temporal axis. Previous studies have shown that 1D CNN models can effectively learn ECG morphology and temporal patterns directly from ECG signals for heartbeat and arrhythmia classification [5–7].

The overall structure of the model contains two main parts: feature extraction and classification head. The feature extraction part consists of four convolutional blocks, which learn temporal features from the ECG signal. Then, GlobalAveragePooling1D converts the temporal feature maps into a compact feature vector. Finally, the classification head uses dense layers, dropout layers, and a sigmoid output to produce the binary prediction.

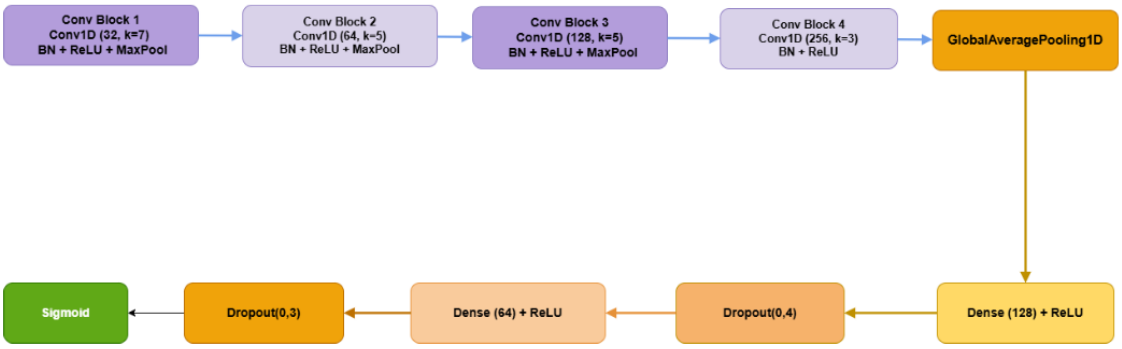


Figure 4.1 Overall architecture of the proposed 1D CNN model

4.2.2. Convolutional Feature Extraction

Although the MIT-BIH Arrhythmia Database contains two-channel ECG records, the CNN experiment in this study uses only the first ECG channel of each record as the input signal. In the implementation, this corresponds to selecting:

$$X = \texttt{record.p_signal[:, 0]}$$

After preprocessing, this single-channel ECG signal is segmented into fixed-length windows and reshaped for the 1D CNN.

For each ECG window, the input shape of the CNN is:

$$(\text{Batch}, n, 1)$$

where n is the number of signal samples in one window. With the MIT-BIH sampling frequency $f_s = 360$ Hz, the value of n depends on the window length:

$$n = f_s \times t$$

For the 4.5-second window:

$$n = 360 \times 4.5 = 1620$$

For the 9-second window:

$$n = 360 \times 9 = 3240$$

Conv Block 1 consists of Conv1D with 32 filters and kernel size 7, Batch Normalization, ReLU activation, and MaxPooling1D. This block reduces the temporal length by half:

$$(\text{Batch}, n, 1) \rightarrow \left(\text{Batch}, \frac{n}{2}, 32\right)$$

Conv Block 2 consists of Conv1D with 64 filters and kernel size 5, Batch Normalization, ReLU activation, and MaxPooling1D:

$$\left(\text{Batch}, \frac{n}{2}, 32\right) \rightarrow \left(\text{Batch}, \frac{n}{4}, 64\right)$$

Conv Block 3 consists of Conv1D with 128 filters and kernel size 5, Batch Normalization, ReLU activation, and MaxPooling1D:

$$\left(\text{Batch}, \frac{n}{4}, 64\right) \rightarrow \left(\text{Batch}, \frac{n}{8}, 128\right)$$

Conv Block 4 consists of Conv1D with 256 filters and kernel size 3, Batch Normalization, and ReLU activation. Unlike the previous three blocks, this block does not use MaxPooling1D, which helps preserve more temporal information before global pooling:

$$\left(\text{Batch}, \frac{n}{8}, 128\right) \rightarrow \left(\text{Batch}, \frac{n}{8}, 256\right)$$

The stacked convolutional blocks allow the model to learn hierarchical ECG representations, starting from low-level local waveform patterns and gradually forming higher-level temporal features. This design follows the general idea of using convolutional layers to automatically extract discriminative representations from ECG signals instead of relying only on handcrafted features [5, 6].

4.2.3. Global Average Pooling 1D

After four convolutional blocks, the output is still a temporal feature map:

$$\left(\text{Batch}, \frac{n}{8}, 256\right)$$

GlobalAveragePooling1D averages each feature map along the temporal dimension. After this step, the tensor is converted into a compact feature vector:

$$(\text{Batch}, 256)$$

This operation summarizes the average activation of each learned pattern across the ECG window, reduces the number of parameters, and helps improve model generalization. Global average pooling is commonly used to replace or reduce fully connected layers in convolutional neural networks [22].

4.2.4. Classification Head

The classification head consists of Dense(128), Dropout(0.4), Dense(64), Dropout(0.3), and one sigmoid output layer:

$$(\text{Batch}, 256) \rightarrow (\text{Batch}, 128) \rightarrow (\text{Batch}, 64) \rightarrow (\text{Batch}, 1)$$

The final output is a value between 0 and 1. A value close to 0 indicates Normal, while a value close to 1 indicates Abnormal.

4.2.5. Training Strategy

Because the task is binary classification between Normal and Abnormal, Binary Cross-Entropy is used as the loss function. Adam is used as the optimizer because it is a common and stable optimization algorithm in deep learning [23]. Class weights are used to reduce the effect of class imbalance.

Early Stopping and ReduceLROnPlateau are also applied. Early Stopping stops the training process when validation performance no longer improves, while ReduceLROnPlateau reduces the learning rate when validation loss does not improve.

4.3. ST-MEM

4.3.1. Motivation

ST-MEM is used in this study as a pre-trained ECG representation learning model. The model is based on masked ECG reconstruction, where parts of the ECG signal are masked and reconstructed during pretraining. This allows the encoder to learn general ECG representations before being transferred to the MIT-BIH downstream classification task [8, 9].

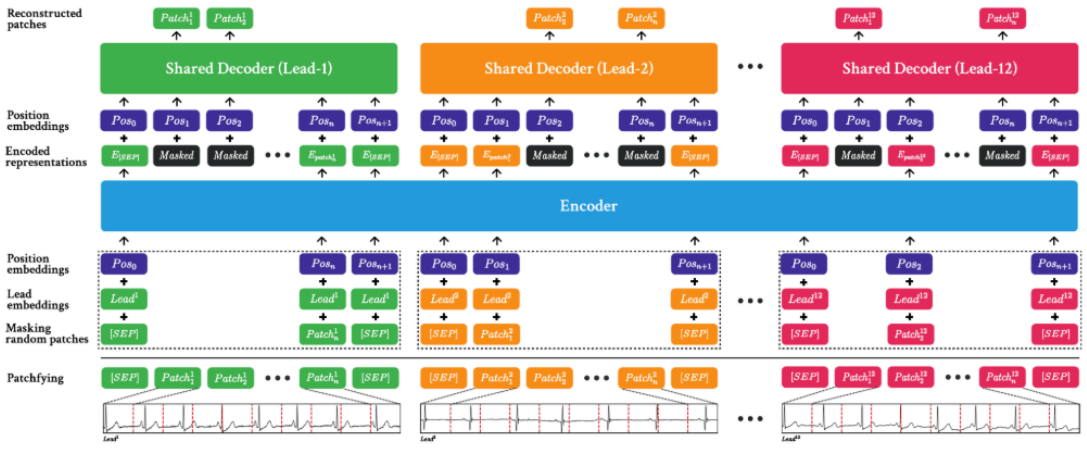


Figure 4.2 ST-MEM architecture

This figure illustrates the original ST-MEM architecture for multi-lead ECG representation learning. In this study, the downstream setting is adapted to the two-lead structure of the MIT-BIH dataset.

4.3.2. Pretrain ST-MEM

In the pretraining stage, ST-MEM learns general ECG representations through a masked reconstruction task. The model randomly masks part of the ECG signal and learns to reconstruct the missing patches. This objective forces the encoder to learn the underlying structure of ECG signals, including waveform morphology, rhythm patterns, temporal relationships, and lead-related information [8].

A sample ECG input is represented as:

$$X \in \mathbb{R}^{B \times L \times T}$$

where B is the batch size, L is the number of leads, and T is the number of time points.

When applying ST-MEM to the MIT-BIH dataset, the ECG signal is resampled to 250 Hz to match the pretraining configuration. Each input sample is fixed to:

$$T = 2250$$

Since MIT-BIH contains two leads, the input of the model becomes:

$$X \in \mathbb{R}^{B \times 2 \times 2250}$$

The model uses the patch size:

$$p = 75$$

Therefore, each lead is divided into:

$$n = \frac{T}{p} = \frac{2250}{75} = 30$$

patches. Each patch contains 75 consecutive signal points. After patchification, the ECG signal is represented as:

$$P \in \mathbb{R}^{B \times 2 \times 30 \times 75}$$

where:

$$P_i^l \in \mathbb{R}^{75}$$

is the i-th patch of lead l.

After patchification, each patch is passed through the patch embedding layer. This layer transforms each raw ECG patch into an embedding vector:

$$E_i^l = PatchEmbed(P_i^l)$$

with:

$$E_i^l \in \mathbb{R}^{768}$$

Then, positional embedding is added to represent the temporal position of the patch in the ECG window:

$$\tilde{E}_i^l = E_i^l + Pos_i$$

where E_i^l is the patch embedding and Pos_i is the positional embedding of patch i. Positional embedding is needed because Transformer-based models do not naturally preserve the order of patches in a sequence.

During pretraining, ST-MEM applies random masking to the patch sequence. With a mask ratio of 0.75, each lead keeps only about 25% of the patches, while most patches are masked. The encoder only receives the visible patches. The masked patches are not directly passed into the encoder; they are represented by mask embeddings only in the decoder stage. This forces the encoder to learn the structure of ECG signals from incomplete information instead of simply copying the input.

After masking, the model adds two SEP embeddings to the beginning and the end of the patch sequence of each lead. For lead l, the token sequence can be written as:

$$[SEP]_{left}^l, \tilde{E}_1^l, \tilde{E}_2^l, \ldots, \tilde{E}_{30}^l, [SEP]_{right}^l$$

In addition to positional embedding and SEP embedding, ST-MEM uses lead embedding to distinguish patches from different ECG leads. With lead embedding, the complete representation of patch i from lead l is:

$$Z_i^l = E_i^l + Pos_i + Lead_l$$

or equivalently:

$$Z_i^l = PatchEmbed(P_i^l) + Pos_i + Lead_l$$

After adding positional embedding, SEP embedding, and lead embedding, the token tensor has the shape:

$$Z \in \mathbb{R}^{B \times 2 \times 32 \times 768}$$

where:

$$32 = 30 + 2$$

This includes 30 real patches and 2 SEP tokens for each lead. The tensor is then flattened before being passed into the Transformer encoder:

$$Z_f \in \mathbb{R}^{B \times 64 \times 768}$$

The encoder of ST-MEM consists of 12 Transformer blocks. With the ViT-base configuration, the encoder uses embedding dimension 768, 12 attention heads, and MLP hidden dimension 3072 [8].

After passing through the encoder, the output tokens become latent representations of the ECG signal:

$$H = Encoder(Z_f)$$

with:

$$H \in \mathbb{R}^{B \times 64 \times 768}$$

During pretraining, the latent representation H is passed into the decoder to reconstruct the masked patches. The decoder reconstructs the ECG patches:

$$\widehat{P} \in \mathbb{R}^{B \times L \times n \times p}$$

The reconstruction loss is calculated only on the masked patches. Let M be the set of masked patches. The pretraining loss is:

$$L_{SSL} = \frac{1}{|M|} \sum_{i \in M} \left\| \widehat{P_i} - P_i \right\|$$

where P_i is the original masked patch, $\widehat{P_i}$ is the reconstructed patch, and M is the set of masked patches.

Through this masked reconstruction process, the encoder learns general ECG representations, including waveform morphology, rhythm structure, temporal information, and lead-related relationships.

4.3.3. Downstream Task on the MIT-BIH Dataset

When transferring ST-MEM to the MIT-BIH classification task, the decoder is no longer used. The decoder is only needed during pretraining for reconstructing masked patches. In the downstream task, this study keeps the pre-trained encoder and adds a new classifier head for binary Normal/Abnormal classification [8, 9].

The preprocessed MIT-BIH signal is passed into the encoder with the shape:

$$X \in \mathbb{R}^{B \times 2 \times 2250}$$

The signal is divided into patches:

$$P \in \mathbb{R}^{B \times 2 \times 30 \times 75}$$

Then, the patches are passed through patch embedding, positional embedding is added, SEP tokens are inserted, lead embedding is added, and the token tensor is flattened into:

$$Z_f \in \mathbb{R}^{B \times 64 \times 768}$$

This token sequence is passed through the 12 Transformer blocks of the encoder:

$$H = Encoder(Z_f)$$

During fine-tuning, random masking and mask embedding are not used. All real ECG patches are passed into the encoder. After the encoder, SEP tokens are removed. The remaining embeddings are averaged along the lead and patch dimensions to create an ECG representation vector:

$$h = Pool(H)$$

where:

$$h \in \mathbb{R}^{B\times 768}$$

This vector is passed through the classifier head to produce the output logit:

$$z = Wh + b$$

Because the MIT-BIH task in this study is binary classification, the output uses the Sigmoid function:

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

The predicted class is determined using a threshold of 0.5:

$$\hat{c} = \begin{cases} 0, & \hat{y} < 0.5 \\ 1, & \hat{y} \geq 0.5 \end{cases}$$

where class 0 represents Normal and class 1 represents Abnormal.

For each ECG sample X_i and label $y_i \in \{0,1\}$, the model predicts:

$$\hat{y}_i = \sigma(Wh_i + b)$$

The loss function is Binary Cross Entropy:

$$L_{BCE} = -\frac{1}{B}\sum_{i=1}^{B}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

Both downstream strategies use the same classifier head with a Sigmoid output and the same Binary Cross Entropy loss for the Normal/Abnormal binary classification task.

Strategy 1: Freeze encoder and train only the classifier head.

In the first strategy, the entire pre-trained encoder is kept frozen. The encoder only works as an ECG feature extractor, while the classifier head is the only component trained on the MIT-BIH dataset.

The processing flow of this strategy can be described as:

$$X \rightarrow Encoder \rightarrow h \rightarrow Classifier \rightarrow \hat{y}$$

In this flow, the *Encoder* has already learned ECG representations during the pretraining stage and is not updated during downstream training. For each ECG input sample, the encoder produces a feature vector h. The classifier head then learns to map this feature vector into the probability of the Normal or Abnormal class.

This strategy is used as a direct evaluation of the representation quality learned by the pre-trained encoder. If the model can achieve good performance by training only the classifier head, it means that the pre-trained encoder has already learned ECG features that can be transferred to the MIT-BIH task. In contrast, if the result is not good, it may indicate that the learned representation is not fully suitable for the characteristics of the MIT-BIH dataset.

In this strategy, the frozen components include patch embedding, positional embedding, SEP embedding, lead embedding, and Transformer blocks. The only trainable component is the classifier head.

Therefore, the freeze encoder strategy is simple, stable, and has a lower risk of overfitting. However, its limitation is that the encoder is not adapted to the specific structure of MIT-BIH, especially because MIT-BIH contains only two leads, while the pretraining data is usually based on multi-lead ECG signals.

Strategy 2: Semi-finetune lead embedding and classifier head.

In the second strategy, the model does not fine-tune the entire encoder. Instead, only the lead embedding and the classifier head are updated during downstream training. The main components of the encoder, such as patch embedding and Transformer blocks, are still kept frozen.

In ST-MEM, each input patch is not represented only by its signal content. It is also combined with positional embedding and lead embedding:

$$Z_i^l = PatchEmbed(P_i^l) + Pos_i + Lead_l$$

where $Lead_l$ helps the model identify which lead the patch belongs to. This component is especially important when applying ST-MEM to MIT-BIH because this dataset contains only two leads. If the lead embeddings from the pre-trained model are kept completely fixed, the model may not adapt well to the lead structure of MIT-BIH.

Therefore, in the semi-finetune strategy, the model is allowed to update the lead embeddings corresponding to the two MIT-BIH leads, together with the classifier head. The trainable components are:

$$Lead_1,\ Lead_2,\ W,\ b$$

where $Lead_1$ and $Lead_2$ are the lead embeddings corresponding to the two leads of MIT-BIH, while W and b are the parameters of the classifier head. The remaining components of the encoder are kept frozen.

The general processing flow is still similar to the first strategy:

$$X \rightarrow Encoder \rightarrow h \rightarrow Classifier \rightarrow \hat{y}$$

However, the key difference is that the model is allowed to adjust the lead embeddings during training. This helps the model adapt better to the MIT-BIH dataset without updating the entire encoder.

The semi-finetune strategy can be seen as an intermediate approach between freezing the encoder and fine-tuning the whole model. Compared with freeze encoder, semi-finetune is more flexible because the model can adjust the representation component directly related to ECG leads. Compared with full fine-tuning, this strategy is safer because fewer parameters are updated, which helps reduce the risk of overfitting on the MIT-BIH dataset.

In summary, the two downstream strategies used in this study are:

- **Freeze encoder:** train only the classifier head.
- **Semi-finetune:** train the lead embedding and classifier head.

In summary, ST-MEM differs from 1D CNN in three main aspects. First, ST-MEM does not learn from scratch on MIT-BIH but uses an encoder pre-trained on a large ECG dataset. Second, instead of using local convolutional kernels, ST-MEM divides the signal into patches and uses self-attention to learn relationships between patches. Third, each patch is combined with positional embedding and lead embedding, allowing the model to exploit both temporal information and lead-related information. This is suitable for ECG signals, which are spatio-temporal signals rather than simple one-dimensional time series [8].

5. Experiments and Results

5.1. Sliding Window

The feature-based sliding-window experiment was evaluated using two window configurations: 4.5 seconds and 9 seconds. The experimental results are shown in Table 5.1. The bold values indicate the best result in each metric group.

Table 5.1 Sliding-window classification results

Model	4.5s				9s			
	Accuracy	**Precision**	**Recall**	**F1-score**	**Accuracy**	**Precision**	**Recall**	**F1-score**
Random Forest	0.5883	0.5271	0.3302	0.4060	**0.7385**	**0.6811**	0.7328	0.7060
Extra Trees	0.5953	0.5464	0.2963	0.3842	0.7300	0.6590	**0.7666**	**0.7087**
XGBoost	**0.6282**	**0.5945**	**0.4016**	**0.4794**	0.6660	0.6103	0.6103	0.6103

The results show that the 9-second window performs clearly better than the 4.5-second window in most evaluation metrics. With the 4.5-second configuration, XGBoost gives the best result among the tested models, achieving the highest Accuracy, Precision, Recall, and F1-score in that window setting. However, its Recall and F1-score are still relatively low, which indicates that the 4.5-second feature-based setting is not strong enough to detect abnormal ECG windows effectively.

With the 9-second configuration, all three models improve significantly. Random Forest with the 9-second window achieves the highest Accuracy and Precision, while Extra Trees with the 9-second window achieves the highest Recall and F1-score. Overall, Extra Trees with the 9-second window is considered the most balanced feature-based model because it detects more abnormal windows and reduces false negatives.

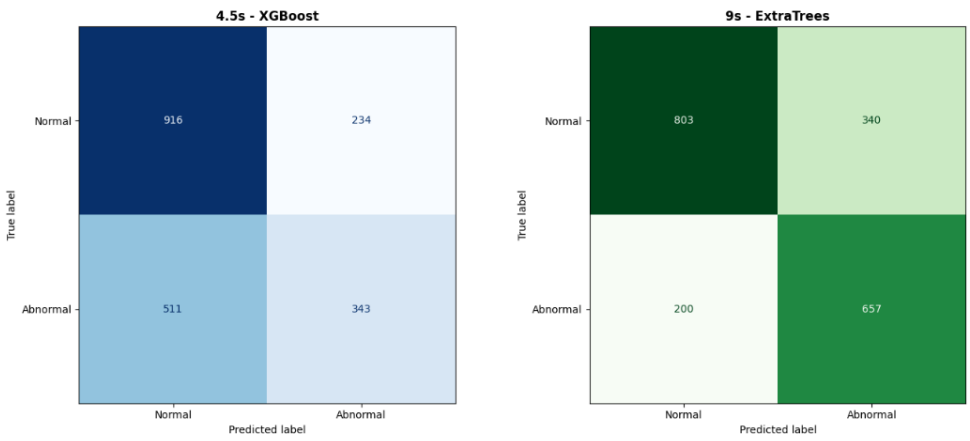


Figure 5.1 Confusion matrix comparison of the best 4.5s and 9s sliding-window models

5.2. CNN

The 1D CNN model was evaluated using two window configurations: 4.5 seconds and 9 seconds. The experimental results are shown in Table 5.2. The bold values indicate the highest value in each metric column.

Table 5.2 CNN classification results

Window	Accuracy	Precision	Recall	F1-score
4.5s	**0.9376**	**0.9409**	0.9400	0.9404
9s	0.9354	0.9354	**0.9554**	**0.9453**

The 4.5-second CNN configuration achieves balanced performance with high Accuracy, Precision, Recall, and F1-score. It obtains the highest Accuracy and Precision among the two CNN settings.

The 9-second CNN configuration achieves the highest Recall and F1-score. This shows that a longer window provides more temporal context, allowing the CNN to detect abnormal ECG windows more effectively. Although the 4.5-second setting has slightly higher Accuracy and Precision, the 9-second setting is better in Recall and F1-score, which are important for abnormal ECG detection.

Overall, the CNN results are much better than the feature-based Sliding Window results. This indicates that the CNN can learn useful ECG patterns directly from the raw preprocessed signal instead of relying only on handcrafted features.

5.3. ST-MEM

The ST-MEM experiment compares two downstream training strategies: Linear Probe and Semi-Finetune. The results are shown in Table 5.3. The bold values indicate the better result between the two strategies for each dataset split.

Table 5.3 ST-MEM classification results

Set	Linear Probe				Semi-Finetune			
	Accuracy	**Precision**	**Recall**	**F1-score**	**Accuracy**	**Precision**	**Recall**	**F1-score**
Validation	0.5415	0.9523	0.6537	0.7752	**0.9920**	**0.9987**	**0.9912**	**0.9949**
Test	0.5392	0.6776	0.8398	0.7500	**0.9672**	**0.9722**	**0.9669**	**0.9695**

The results show that Semi-Finetune performs much better than Linear Probe on both validation and test sets. With Linear Probe, the pre-trained encoder is frozen and only the classifier head is trained. This strategy reaches an F1-score of 0.7500 on the test set. Its Recall is relatively high at 0.8398, but its Accuracy is only 0.5392 and Precision is 0.6776. This indicates that the model can detect many abnormal samples, but it also produces many incorrect predictions.

One reason why Linear Probe is weaker is the difference between the original ST-MEM setting and the MIT-BIH dataset. ST-MEM is commonly designed for multi-lead ECG representation learning, while MIT-BIH contains only two ECG leads. When the encoder is fully frozen, the model has limited ability to adapt its learned lead-related representation to the two-lead structure of MIT-BIH.

With Semi-Finetune, ST-MEM achieves very strong results on both validation and test sets. On the test set, it reaches Accuracy = 0.9672, Precision = 0.9722, Recall = 0.9669, and F1-score = 0.9695. This improvement is mainly related to the reduced-lead adaptation mechanism in the semi-finetune strategy. By updating the lead embedding and classifier head, the model can better adapt to the two-lead MIT-BIH input while still preserving the general ECG representation learned during pretraining.

As a supplementary comparison, to further demonstrate the difference between ST-MEM and a conventional supervised neural network, an additional experiment was conducted on another ECG dataset, PTB-XL [24, 25]. PTB-XL is a large public 12-lead ECG dataset, which makes it more suitable for evaluating models originally designed for multi-lead ECG representation learning.

In this setting, the supervised baseline uses the same ViT-B encoder architecture as ST-MEM [26]. However, unlike ST-MEM, it is initialized with random weights and trained directly on the labelled downstream data without masked ECG pretraining. In contrast,

ST-MEM first uses masked reconstruction to pre-train the encoder, and then the whole model is fine-tuned on the labelled downstream data.

Table 5.4 Comparison between supervised training and ST-MEM on PTB-XL

Model	Accuracy	F1-score
Supervised	0.787	0.604
ST-MEM Linear Probe	0.726	0.508
ST-MEM Finetune	**0.825**	**0.655**

The PTB-XL comparison shows that ST-MEM Finetune outperforms the supervised ViT-B baseline in both Accuracy and F1-score. This result suggests that masked ECG pretraining helps the model learn useful ECG representations before supervised training. However, ST-MEM Linear Probe performs worse than the supervised baseline, indicating that directly using a frozen encoder is not always sufficient. The model benefits more when the pre-trained representation is adapted through full fine-tuning.

5.4. Comparison Design

To compare the three approaches, the best result from each method group is selected on the same record-level test split.

- **Sliding Window:** the best Extra Trees result on the test set.
- **CNN:** the best 1D CNN result on the test set.
- **ST-MEM:** the Semi-Finetune result on the test set.

Table 5.5 Best model comparison among Sliding Window, CNN, and ST-MEM on the test set

Method	Accuracy	Precision	Recall	F1-score
Sliding Window – Extra Trees	0.7300	0.6590	0.7666	0.7087
1D CNN	0.9354	0.9354	0.9554	0.9453
ST-MEM – Semi-Finetune	**0.9672**	**0.9722**	**0.9669**	**0.9695**

When comparing the best results of the three approaches on the test set, Sliding Window with Extra Trees achieves an F1-score of 0.7087, 1D CNN achieves an F1-score of 0.9453, and ST-MEM Semi-Finetune achieves an F1-score of 0.9695.

The results show that the Sliding Window approach has the lowest performance among the three methods. Although Extra Trees is the best configuration in the feature-based group, it still depends heavily on handcrafted features such as RR intervals, heart rate statistics, and frame-level signal features. These handcrafted features can describe some ECG characteristics, but they may not fully capture complex abnormal patterns.

The 1D CNN model performs much better than the Sliding Window approach. With an F1-score of 0.9453, CNN demonstrates the ability to learn ECG features directly from the signal. This is more effective than relying only on manually designed features.

Among the three methods, ST-MEM Semi-Finetune achieves the best performance, with Accuracy = 0.9672, Precision = 0.9722, Recall = 0.9669, and F1-score = 0.9695. This model not only achieves the highest F1-score but also maintains a strong balance between Precision and Recall.

Overall, the performance increases in the following order:

$$\text{Sliding Window} < \text{1D CNN} < \text{ST-MEM Semi-Finetune}$$

This result suggests that deep learning methods, especially pre-trained ECG representation models such as ST-MEM, have a clear advantage in ECG Normal/Abnormal classification.

6. Discussion, Limitations, and Conclusion

6.1. Discussion

The experimental results show that model performance improves when the method has a stronger ability to learn ECG signal representations. In the Sliding Window approach, handcrafted features such as RR intervals, heart rate statistics, and frame-level features can support ECG classification. However, this approach still has limitations in representing complex abnormal ECG patterns.

Compared with the Sliding Window approach, the 1D CNN model achieves clearly better performance because it can learn features directly from ECG signals. The model can exploit important information such as QRS morphology, amplitude, slope, and local waveform changes without depending heavily on handcrafted feature extraction.

Among the tested methods, ST-MEM Semi-Finetune is the most effective model. This result shows that a pre-trained model has an advantage in learning general ECG representations. At the same time, the semi-finetune strategy helps the model adapt better to the MIT-BIH dataset.

6.2. Limitations

Although the results are promising, this study still has several limitations. First, the MIT-BIH Arrhythmia Database contains only 48 ECG records, so the number of patients and the diversity of the data are limited. This may affect the generalization ability of the models when they are applied to real-world ECG data.

Second, the task is simplified into binary Normal/Abnormal classification, while real cardiac arrhythmias include many different types. Therefore, the current results only reflect the ability to detect general abnormality and do not evaluate the ability to distinguish specific arrhythmia classes.

Third, ST-MEM was originally developed for multi-lead ECG data, usually 12-lead ECG, while the MIT-BIH dataset contains only two leads. Therefore, when using Linear Probe and keeping the encoder fully frozen, the model may not adapt well to the new data structure, leading to limited performance. However, this is also one of the strengths of ST-MEM because the model has a mechanism to adapt to reduced-lead ECG settings.

When Semi-Finetune is applied, especially by allowing the lead embedding to be updated, the model can learn a representation that is more suitable for the two-lead MIT-BIH data. As a result, ST-MEM Semi-Finetune achieves superior performance compared with Linear Probe and the other tested models.

6.3. Conclusion

The main contribution of this study is that it builds and compares multiple approaches on the same ECG Normal/Abnormal classification task. Through this comparison, the study shows that suitable preprocessing, an appropriate windowing strategy, and model selection have a strong influence on classification quality.

The results also suggest that pre-trained models such as ST-MEM are a promising direction for ECG classification tasks, especially when labelled data are limited. By using pre-trained ECG representations and adapting them through semi-finetuning, the model can achieve strong performance on the MIT-BIH dataset.

In future work, this study can be extended in several directions. First, the task can be developed from binary classification to multi-class classification to identify specific types of arrhythmia. Second, the models should be evaluated on additional ECG datasets to examine their generalization ability. Third, more fine-tuning strategies for ST-MEM can be tested, hyperparameters can be optimized, and model complexity can be reduced to make the approach more suitable for real-world deployment.

REFERENCES

[1] Y. Sattar and L. Chhabra, "Electrocardiogram," StatPearls, NCBI Bookshelf, 2023, available at NCBI Bookshelf.

[2] G. B. Moody and R. G. Mark, "Mit-bih arrhythmia database," PhysioNet, 2005.

[3] ——, "The impact of the mit-bih arrhythmia database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.

[4] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "Physiobank, physiotoolkit, and physionet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[5] S. Kiranyaz, T. Ince, and M. Gabbouj, "Real-time patient-specific ecg classification by 1-d convolutional neural networks," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016.

[6] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Adam, A. Gertych, and R. S. Tan, "A deep convolutional neural network model to classify heartbeats," *Computers in Biology and Medicine*, vol. 89, pp. 389–396, 2017.

[7] O. Yildirim, P. Plawiak, R. S. Tan, and U. R. Acharya, "Arrhythmia detection using deep convolutional neural network with long duration ecg signals," *Computers in Biology and Medicine*, vol. 102, pp. 411–420, 2018.

[8] Y. Na, M. Park, Y. Tae, and S. Joo, "Guiding masked representation learning to capture spatio-temporal relationship of electrocardiogram," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=WcOohbsF4H

[9] VUNO Inc., "St-mem: Spatio-temporal masked electrocardiogram modeling," https://github.com/vuno/ST-MEM, 2024, official implementation of "Guiding Masked Representation Learning to Capture Spatio-Temporal Relationship of Electrocardiogram". Accessed: 2026-05-12.

[10] N. Emanet, "Ecg beat classification by using discrete wavelet transform and random forest algorithm," 2009, reference listed in the project report.

[11] C. V. Banupriya and S. K. Saravanan, "Electrocardiogram beat classification using support vector machine," 2014, reference listed in the project report.

[12] T. J. Jun, H. M. Nguyen, D. Kang, D. Kim, D. Kim, and Y.-H. Kim, "Ecg arrhythmia classification using a 2-d convolutional neural network," *arXiv preprint arXiv:1804.06812*, 2018.

[13] J. Pan and W. J. Tompkins, "A real-time qrs detection algorithm," *IEEE Transactions on Biomedical Engineering*, vol. BME-32, no. 3, pp. 230–236, 1985.

[14] G. D. Clifford, F. Azuaje, and P. E. McSharry, Eds., *Advanced Methods and Tools for ECG Data Analysis*. Artech House, 2006.

[15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[16] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[17] P. de Chazal, M. O'Dwyer, and R. B. Reilly, "Automatic classification of heartbeats using ecg morphology and heartbeat interval features," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004.

[18] tsfresh Developers, "Feature extraction settings," https://tsfresh.readthedocs.io/en/latest/text/feature_extraction_settings.html, 2026, accessed: 2026-05-12.

[19] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[20] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, pp. 3–42, 2006.

[21] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[22] M. Lin, Q. Chen, and S. Yan, "Network in network," *arXiv preprint arXiv:1312.4400*, 2013.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[24] P. Wagner, N. Strodthoff, R.-D. Bousseljot, D. Kreiseler, F. I. Lunze, W. Samek, and T. Schaeffter, "Ptb-xl, a large publicly available electrocardiography dataset," *Scientific Data*, vol. 7, no. 1, p. 154, 2020.

[25] ——, “Ptb-xl, a large publicly available electrocardiography dataset, version 1.0.3,” PhysioNet, 2022.

[26] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021. [Online]. Available: https://arxiv.org/abs/2010.11929